# Memory-Bounded Teacher Scoring

FSDP2 readout optimization and validation


| | |
|---|---|
| Author | Jiacheng Yu |
| System | Qwen2.5-0.5B-Instruct, BF16, one GPU |
| Execution | verl Teacher Worker, FSDP2 with CPU offload |


**Principal result.** At 8,192 projected rows, peak allocated during the Worker call decreased from 3,596.6 to 681.6 MiB, while candidate logprobs and ordered Teacher Top-$K$ matched the dense reference. The allocation measurement is scoped to the PyTorch allocator.

# Abstract

Teacher scoring can return compact outputs while first materializing a much larger row-by-vocabulary logits tensor. This project integrates a row-chunked LM-head readout into a pinned verl `TeacherFSDPWorker`. It preserves full-vocabulary-normalized log probabilities at Student candidate IDs and the Teacher's ordered Top-$K$, while entering through the normal FSDP2 forward and retaining tied weights and module hooks.

For Qwen2.5-0.5B-Instruct in BF16 on one RTX 5070 Laptop GPU, with $K = 32$, 256-row chunks, CPU offload and allocator fraction 0.50, the $N = 8192$ Worker comparison reduced peak allocated by 81.05%. All 262,144 candidate scores and 8,192 ordered Top-$K$ rows matched exactly. A separate direct Worker-to-Student fixture matched the pinned loss, checked gradients and one restricted RMSNorm update. The result is a single-GPU experimental optimization; the measured timings are descriptive.


# Contents


**1 Problem and scoring contract** **2**

**2 System architecture** **2**

**3 Implementation** **3**

3.1 FSDP and the LM-head lifecycle . . . . . 3

3.2 Chunk selection and dense fallback . . . . . 3

3.3 Padding and consumer alignment . . . . . 3

**4 Experimental setup** **3**

**5 Results** **4**

5.1 Allocator peaks and capacity . . . . . 4

5.2 Timing interpretation . . . . . 4

**6 Worker-to-Student validation** **4**

**7 Current scope and future work** **5**

**8 Reproducibility** **5**

**9 Conclusion** **5**

# 1 Problem and scoring contract

Let $N$ be the number of projected model rows, $V$ the vocabulary size and $K$ the candidate width. A dense language-model head can allocate $N \times V$ logits even when the response contains only $O(NK)$ values and IDs. Reducing the response alone therefore does not bound the intermediate readout allocation.

For hidden row $h_t$, head weights $W$, bias $b$ and candidate $i$, the required full-vocabulary-normalized score is

$$z_{t,i} = h_t^{\mathsf{T}} W_i + b_i, \qquad \log p_{t,i} = z_{t,i} - \log \sum_{j \in \mathcal{V}} e^{z_{t,j}}. \tag{1}$$

The denominator covers the entire vocabulary. The Worker also returns the Teacher's own ordered Top-$K$ IDs, so projecting only Student candidate logits cannot reproduce the complete interface in general. Candidate IDs can repeat; their order and multiplicity, valid-row order and nested offsets are part of the output contract.

Row chunking and selected-logit primitives are established techniques. The engineering contribution here is their integration with the fixed Worker, FSDP lifecycle and downstream scoring contract.[1–3]

# 2 System architecture

The adapter is opt-in. Dense and chunked paths enter the same Worker and return the same compact outputs; the last stage in Figure 1 is a separate consumer-validation fixture.

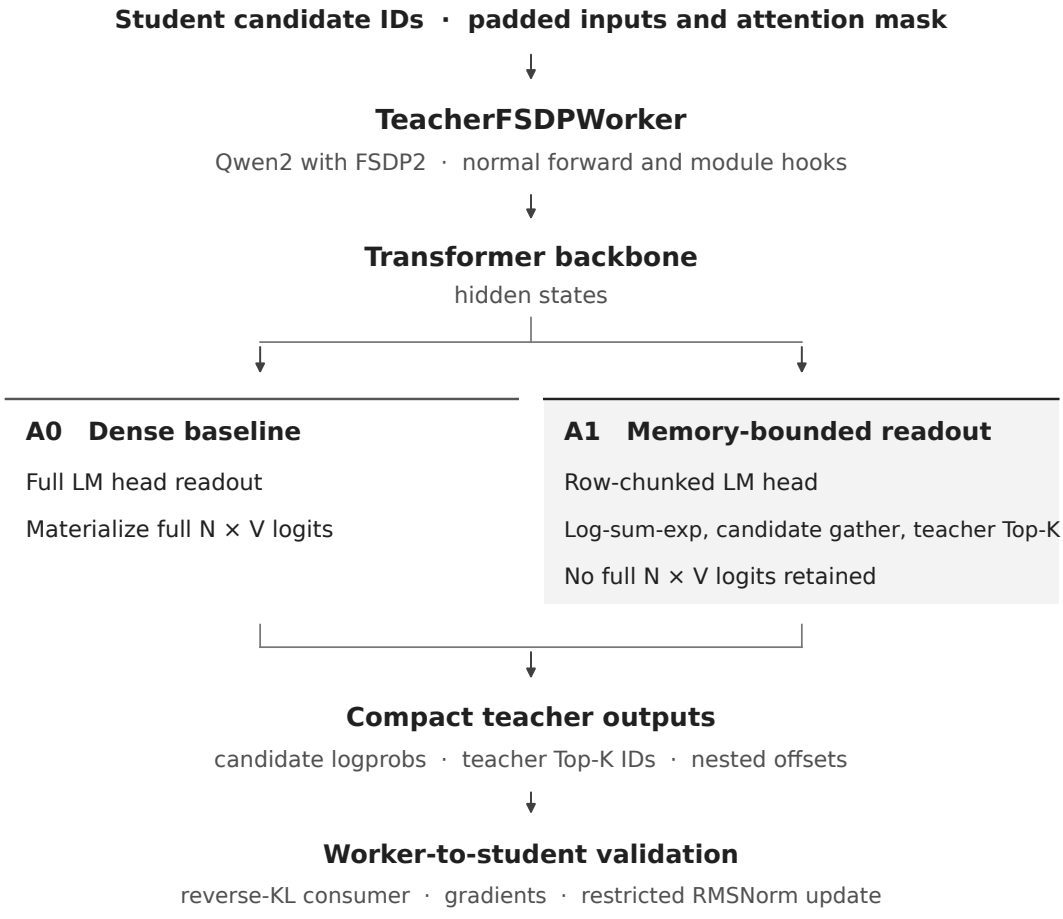



**Figure 1.** Teacher scoring paths and the restricted Worker-to-Student validation.

# 3 Implementation

## 3.1 FSDP and the LM-head lifecycle

The adapter is installed before wrapping the ordinary `Qwen2ForCausalLM`; it retains the parameter objects and the tied input/output weight. A request enters the normal outer FSDP module call. The backbone and final normalization run once, and the resulting hidden rows are passed through `lm_head` as a module, in tiles. This preserves the head's pre- and post-hooks as well as the outer FSDP forward boundary.

For each tile, logits are promoted to FP32 for log-sum-exp and candidate gather; the selected scores are cast back to the logits dtype and the Teacher Top-$K$ IDs are retained. The tile is discarded before the next one. The live logits tile therefore scales with $BV$ rather than $NV$ for $B < N$; compact outputs still scale with $NK$, and the model, backbone and FSDP allocations remain. Full-vocabulary projection work is not eliminated.

## 3.2 Chunk selection and dense fallback

An optional head budget selects the tile size before the forward. If $s$ is the logits element size in bytes, $H$ the requested readout budget, and $B_{\text{requested}}$ the requested row cap (or $N$ if omitted), the implementation's estimate is

$$M_c = NK(s+8), \qquad m_r = \lceil 1.25\,[V(s+8) + 24K] \rceil\,, \tag{2}$$

$$B = \min\left(N, B_{\text{requested}}, \left\lfloor \frac{H - M_c}{m_r} \right\rfloor\right). \tag{3}$$

The estimate includes compact outputs, a logits tile, FP32 work tensors and Top-$K$ bookkeeping; it does not bound all model allocations, backend workspaces or fragmentation. If no explicit budget is set and the selected tile covers all rows, the integration uses the native dense forward. An explicit budget does not silently select that fallback.

## 3.3 Padding and consumer alignment

Candidate IDs are scattered into padded model rows using the prepared attention mask, and returned valid rows are restored in their original order. The adapter does not add a causal shift to those IDs. In the consumer check, the response mask is aligned to prediction positions before aggregate loss and diagnostic calculations.

# 4 Experimental setup

The final comparison used Qwen2.5-0.5B-Instruct, BF16 and SDPA on one RTX 5070 Laptop GPU, with PyTorch 2.14.1+cu130, FSDP2 CPU offload, $K = 32$, $B = 256$ and PyTorch allocator fraction 0.50. Text and candidate IDs were synthetic. For each length, four fresh processes ran in Dense–Chunked–Chunked–Dense order. Each process had one warmup and five measured calls; each p50/p95 below summarizes ten calls per mode.

All 96 saved compact outputs from the final matrix and the chunk-size sweep were reopened and compared against same-length Dense references. At $N = 8192$, all 262,144 candidate logprobs and all 8,192 ordered Teacher Top-$K$ rows matched exactly; the comparison also checked output identity, shapes and offsets.

# 5 Results

## 5.1 Allocator peaks and capacity

Table 1 records per-call PyTorch peaks and synchronized Worker spans. At $N = 8192$, peak allocated fell from 3,596.6 MiB to 681.6 MiB, a reduction of 81.05%. Reserved peaks are reported separately. The measurement is not a whole-device VRAM reduction.

**Table 1.** Worker-call allocator peaks and warm elapsed time.

| $N$ | Mode | Alloc. MiB | Reserved MiB | p50 ms | p95 ms |
|---|---|---|---|---|---|
| 1024 | Dense | 1517.1 | 2644 | 210.4 | 288.0 |
| 1024 | Chunked | 665.3 | 1124 | 159.4 | 187.0 |
| 4096 | Dense | 2409.4 | 2944 | 322.3 | 414.4 |
| 4096 | Chunked | 672.3 | 1130 | 318.8 | 327.5 |
| 8192 | Dense | 3596.6 | 3846 | 570.8 | 599.6 |
| 8192 | Chunked | 681.6 | 1126 | 626.3 | 944.6 |

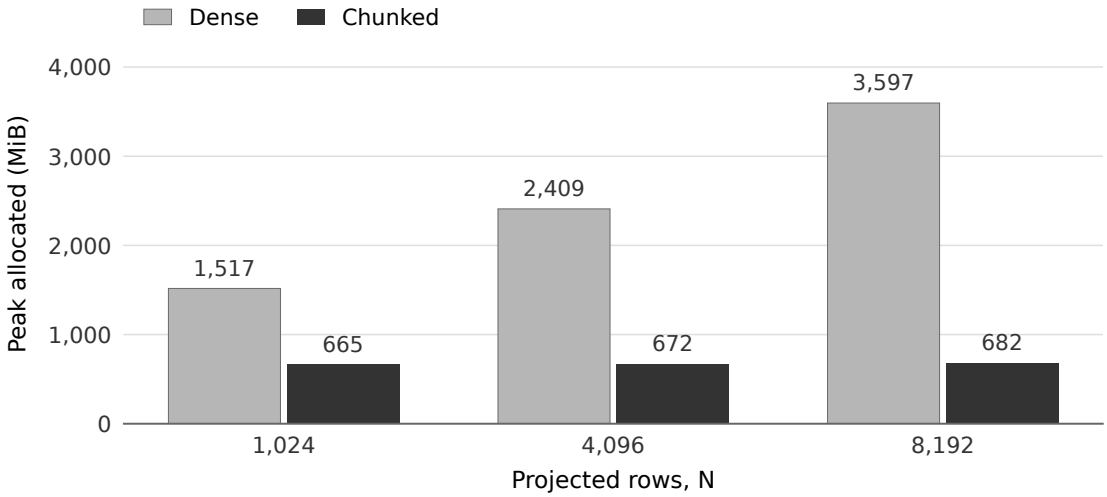



**Figure 2.** Peak allocated for the Table 1 conditions; bar labels are rounded to the nearest MiB.

At fraction 0.50, Dense succeeded at 8,704 rows and reached allocator OOM at 9,216; Chunked ran at 32,768, the largest tested point. These are tested points rather than exact capacity limits, and the 32,768-row run has no same-length Dense correctness reference.

## 5.2 Timing interpretation

The synchronized spans include diagnostic hooks and nested-output return to CPU. At 8,192 rows, Dense p50/p95 was 570.8/599.6 ms and Chunked was 626.3/944.6 ms. The small sample showed process variation and host-clock differences, so it does not establish a general latency improvement.

# 6 Worker-to-Student validation

In a separate two-sequence fixture, actual Dense and Chunked Worker outputs were passed directly to the pinned reverse-KL consumer. The fixture used $K = 8$, 25 active response-prediction rows and one AdamW update restricted to the Student's final RMSNorm weight. Table 2 shows selected observations; the checked Student-logits gradients and final RMSNorm gradients also matched exactly.

**Table 2.** Direct Worker-to-Student fixture.

| Quantity | Dense | Chunked |
|---|---|---|
| Masked scalar loss | 0.0216006283 | 0.0216006283 |
| Final RMSNorm gradient L2 | 0.0124774044 | 0.0124774044 |
| Changed BF16 parameter values | 1 | 1 |

An earlier consumer implementation matched the scalar loss but differed in the checked gradients. Reusing the FP32 conversion as in the pinned consumer removed that difference in the recorded fixture. This illustrates why the validation compares gradients and the update as well as the scalar loss.

## 7 Current scope and future work

The measured path covers a tied-head Qwen2 model on one GPU with FSDP2; the consumer experiment is a restricted update. Future work includes packed variable-length inputs, broader model coverage, FSDP1 and multi-GPU/tensor-parallel validation, and end-to-end RL Trainer integration. The earlier selected-logit-only fusion proposal remains NO-GO for the complete Teacher contract; its negative findings remain in the historical results.

## 8 Reproducibility

The evaluated scoring baseline is `ad395e9`. The Worker patch targets verl `87cd055e872f9fdd7b976fcd797a0934b6dba265`; the pinned Qwen model revision is `7ae557604adf67be50417f59c2c2f167def9a775`. The dedicated Linux/Python 3.12 Worker lock fixes 105 dependencies. The reproduction guide supplies the environment, model hashes and commands; the evidence manifest binds archived measurements by SHA-256.

The GitHub CPU regression run at presentation revision `c0f019e` reported 171 passed, two deselected and no skipped tests. It checks CPU regressions and patch applicability; GPU Worker measurements are a separate experiment.

## 9 Conclusion

Moving the row boundary ahead of the LM head reduces the live readout allocation while preserving the compact scoring outputs. The measured single-GPU result, exact output comparisons and restricted Student update provide a concrete basis for further integration.

## References

[1] Jordan Pettyjohn. IndexedLogits: indexed selected-logit CUDA implementation, 2026. URL `https://github.com/pettyjohnjn/indexed_logits/tree/6020cb446cde8cb2b74a8fdf90db905a320936c3`. Source inspected 2026-10-28; no project runtime reproduction.

[2] SGLang contributors. PR 38119: Add sparse per-position requested-token scoring, 2026. URL `https://github.com/sgl-project/sglang/pull/38119`. Source inspected 2026-10-28; no project runtime reproduction.

[3] verl contributors. Teacher scoring and student-top-K distillation, PR 6920, 2026. URL `https://github.com/verl-project/verl/pull/6920`. Experiments use source revision 87cd055e872f9fdd7b976fcd797a0934b6dba265 and a local project patch; no upstream acceptance is implied.